

Iterated Closest Point (ICP) for Scan Matching in SLAM Applications

Oliver MacDonald

April 30, 2025

Abstract

This paper investigates the mathematical foundations, implementation, and evaluation of Iterative Closest Point (ICP) algorithms for scan matching, a core component of SLAM (Simultaneous Localization and Mapping) in autonomous robotic systems. Variants including point-to-point, point-to-plane, and Generalized ICP are examined with respect to accuracy, convergence, and computational cost in the context of edge device deployment for lunar robotics.

1 Introduction

Autonomous navigation has become a major focus of modern robotics, especially in environments where external positioning systems like GPS are unavailable. This challenge is emphasized by the NASA Lunabotics competition, in which Boise State University’s Bender Robotics Team is competing with their fully autonomous lunar excavation robot, *Bender*. The robot is tasked with navigating rough, dusty terrain, collecting regolith, and returning it to a specified location—entirely without human intervention.

Bender employs a multi-sensor architecture that includes 3D LiDAR, stereo cameras, an IMU, and a Vive beacon system for indoor positional tracking. Despite this advanced suite, the navigation stack has exhibited failure modes, particularly in odometry and mapping. These failures manifest as “tornado mode,” in which accumulated error causes the map to rotate around the robot, resulting in incorrect obstacle detection and incorrect positioning. This ultimately forces the robot to make incorrect navigation decisions, such as turning in place indefinitely or abandoning its planned path.

In GPS-denied environments like the lunar surface, correcting this drift without an external reference requires accurate scan matching between consecutive LiDAR frames. Iterative Closest Point (ICP) algorithms offer a compelling solution, using geometric relationships between overlapping point clouds to estimate the robot’s motion and update the map. However, ICP comes with trade-offs between computational cost and alignment accuracy, which are especially relevant for embedded platforms like the ESP32, Teensy 4.1, and Raspberry Pi 5, where real-time constraints demand μ 10ms compute cycles.

The objective of this work is to analyze and compare three major variants of the ICP algorithm being Point-to-Point, Point-to-Plane, and Generalized ICP. This paper focuses on their mathematical formulation, numerical stability, and suitability for real-time implementation on constrained hardware. The aim is to determine which variant provides the best balance of performance and computational cost for potential deployment on the Bender robot during the final testing phase.

The project contribution is twofold:

- A structured evaluation of ICP variants using both Python prototypes and C-based embedded implementations.

- A set of recommendations for deploying accurate, lightweight scan matching algorithms for real-time SLAM on edge robotics platforms.

2 ICP Algorithm Variants

2.1 Point-to-Point ICP

Point-to-point ICP seeks the transformation $T = (R, \mathbf{t})$ that minimizes the sum of squared Euclidean distances between corresponding points in the source and target point clouds:

$$E(R, \mathbf{t}) = \sum_{i=1}^N \|\mathbf{q}_i - (R\mathbf{p}_i + \mathbf{t})\|^2$$

This approach assumes that all point correspondences are valid and carries the risk of local minima when the overlap is poor. The optimal transformation is solved via SVD on the cross-covariance matrix between centered point clouds [Donoso et al., 2017].

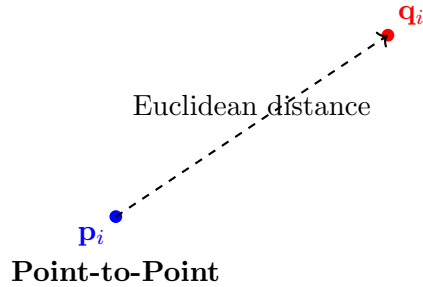


Figure 1: Point-to-Point ICP minimizes direct Euclidean distance between corresponding points.

2.2 Point-to-Plane ICP

Point-to-plane ICP leverages local surface normals to minimize the distance from each source point to the tangent plane at the corresponding target point. The objective function becomes:

$$E(R, \mathbf{t}) = \sum_{i=1}^N \left[\mathbf{n}_i^\top (\mathbf{q}_i - (R\mathbf{p}_i + \mathbf{t})) \right]^2$$

This leads to a linear least squares system solved via QR decomposition. It tends to converge faster than point-to-point ICP, especially when surface geometry is well-defined [Guevara et al., 2024].

2.3 Generalized ICP

Generalized ICP (G-ICP) fuses point-to-point and point-to-plane ideas into a probabilistic framework. Each point is modeled as a Gaussian with associated covariance:

$$E(T) = \sum_{i=1}^N (\mathbf{q}_i - T\mathbf{p}_i)^\top (C_q + RC_p R^\top)^{-1} (\mathbf{q}_i - T\mathbf{p}_i)$$

This formulation improves robustness to noise and outliers by weighting residuals based on local uncertainty. It reduces sensitivity to poor correspondences and unifies multiple ICP variants under a common model [Segal et al., 2009].

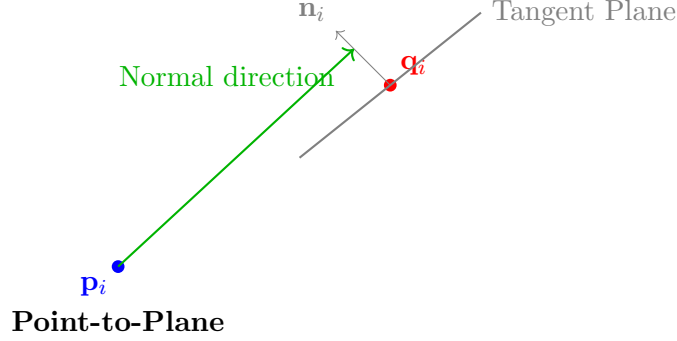


Figure 2: Point-to-Plane ICP minimizes distance along the normal vector of the target surface.

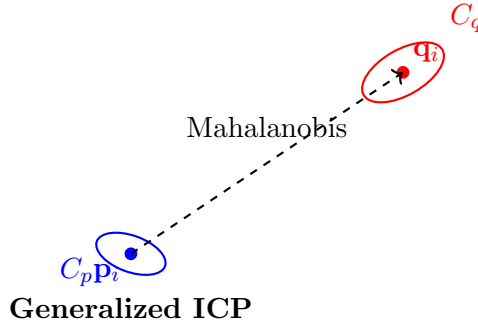


Figure 3: G-ICP minimizes Mahalanobis distance using local point covariances.

2.4 Comparison Framework

To assess the practical trade-offs of ICP variants, we compare them using:

- **Metrics:** Accuracy (final alignment error), Precision (variance over trials), and Runtime (ms/frame)
- **Datasets:** Both synthetic point clouds (e.g., shapes, noisy transforms) and real-world terrain data
- **Target Platforms:** Raspberry Pi 5, Teensy 4.1, and ESP32 for embedded deployment constraints

3 Formulation

3.1 Rigid Transformations in 3D

A rigid body transformation T in 3D applies a rotation $R \in SO(3)$ and translation $\mathbf{t} \in \mathbb{R}^3$ to a point $\mathbf{p} \in \mathbb{R}^3$:

$$T(\mathbf{p}) = R\mathbf{p} + \mathbf{t}$$

Rigid transformations preserve distances and angles, which is crucial for physical scan alignment tasks.

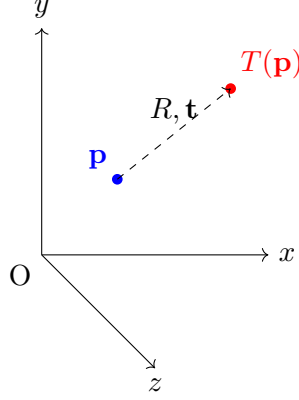


Figure 4: Rigid transformation in 3D showing rotation R and translation $\mathbf{t}$.

3.2 Point Cloud Registration

Given a source point cloud $\mathcal{P} = \{\mathbf{p}_i\}$ and a target (map) point cloud $\mathcal{Q} = \{\mathbf{q}_i\}$, ICP attempts to find the transformation $T = (R, \mathbf{t})$ that minimizes alignment error:

$$T^* = \arg \min_{R, \mathbf{t}} \sum_i \|\mathbf{q}_i - (R\mathbf{p}_i + \mathbf{t})\|^2$$

This assumes point correspondences $(\mathbf{p}_i, \mathbf{q}_i)$ are known or estimated via nearest neighbor search. Variants modify the error metric, correspondence criteria, and optimization strategy.

3.3 Least Squares & SVD for Best-Fit Transform

For point-to-point ICP, once correspondences are determined, we solve for T^* by minimizing the Frobenius norm of residuals. Centering both point sets (by subtracting their means) leads to an orthogonal Procrustes problem:

$$\min_R \|R\mathbf{P}_c - \mathbf{Q}_c\|_F^2$$

This is solved using Singular Value Decomposition (SVD) on the cross-covariance matrix $H = \sum_i \mathbf{p}_i \mathbf{q}_i^\top$. Let $H = U\Sigma V^\top$, then $R = VU^\top$ ensures the closest orthogonal matrix through a least squares approach.

3.4 Overdetermined Systems & QR Decomposition

For point-to-plane ICP, we minimize the orthogonal distance from a transformed point to the tangent plane of its correspondence:

$$E = \sum_i \left[\mathbf{n}_i^\top (\mathbf{q}_i - R\mathbf{p}_i - \mathbf{t}) \right]^2$$

This is linearized using small-angle approximations and solved via the least squares method:

$$Ax = b \quad \text{where} \quad A \in \mathbb{R}^{N \times 6}, \quad x = [\delta\theta_x, \delta\theta_y, \delta\theta_z, t_x, t_y, t_z]^\top$$

The overdetermined system is efficiently solved with QR decomposition or normal equations.

3.5 Covariance & Probabilistic Models in G-ICP

In Generalized ICP, each point in the source and target cloud is treated as a Gaussian-distributed random variable:

$$\mathbf{p}_i \sim \mathcal{N}(\hat{\mathbf{p}}_i, C_p), \quad \mathbf{q}_i \sim \mathcal{N}(\hat{\mathbf{q}}_i, C_q)$$

The goal is to minimize the Mahalanobis distance between transformed source points and their targets:

$$T^* = \arg \min_T \sum_i (\mathbf{q}_i - T\mathbf{p}_i)^\top (C_q + RC_p R^\top)^{-1} (\mathbf{q}_i - T\mathbf{p}_i)$$

The covariance matrices C_p and C_q describe local surface uncertainty, estimated via PCA in a local neighborhood.

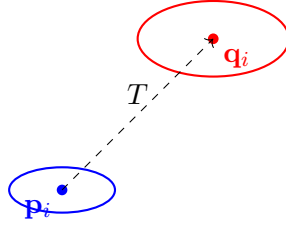


Figure 5: Covariance ellipses for a source-target pair in G-ICP. Uncertainty is modeled via local PCA.

Unlike standard ICP, G-ICP weighs the alignment based on both structural orientation and measurement noise, resulting in a more robust and symmetric scan matching framework [Segal et al., 2009].

4 Implementation

Both Python and C++ versions were created to support rapid development, benchmarking, and eventual deployment on embedded hardware platforms. The framework supports Point-to-Point, Point-to-Plane, and Generalized ICP variants.

4.1 Python Implementation Overview

The Python implementation is designed for modularity and experimentation. A central `icp.py` module defines a base `ICPAlgorithm` class, which is subclassed by `PointToPointICP`, `PointToPlaneICP`, and `GeneralizedICP`. Each variant follows a consistent API, supporting plug-and-play benchmarking across different datasets.

Point correspondence is handled using multiple nearest neighbor strategies implemented via `scikit-learn`'s `NearestNeighbors`. Local normals and covariances are estimated using PCA on k-NN neighborhoods. Visualization is performed using `matplotlib` for 2D cases and `plotly` for 3D scenes.

Configurations for each run are specified using a custom `ICPConfig` data class:

- **Segmentation:** `none`, `even_points`, `even_range`, `uneven_points`
- **Filtering:** Dropout percentage via Gaussian noise filtering
- **Memory Modes:** `full`, `previous`, `sliding` (controls map history size)

- **Rejection Criteria:** Max distance, normal difference threshold, percentile clipping
- **Neighborhood Search:** fixed radius or `density_adaptive`
- **Optional:** Number of map segments, density factors

This structure enables reproducible experimentation and makes it easy to test new matching, filtering, or segmentation strategies.

4.2 C/PlatformIO Version Overview

The C++ implementation is designed for deployment on embedded systems and is integrated into a larger SLAM system currently under development. It uses the PlatformIO build system to target both the Teensy 4.1 and ESP32 microcontrollers.

All three ICP variants are implemented in C++ using the **Eigen** linear algebra library to support efficient matrix and vector operations. The ICP module is class-based and can be compiled independently or linked into larger robotic frameworks. The system is compatible with fixed-point math and optimized for deterministic behavior under low-power, real-time constraints.

4.3 Edge Deployment Strategy

The ultimate goal of this work is to identify which ICP variant best balances runtime and accuracy on constrained hardware. For this reason, the system is designed to operate within a strict <10 ms processing window per frame.

Before scan matching, all sensor data (LiDAR, IMU, etc.) is merged into a shared spatial grid using fixed-point math. This preprocessing ensures that the ICP module operates on a consistent input space. The resulting transformation estimates are streamed via serial to a base station or visualized in real-time for debugging.

Variant selection is currently manual, with this research aimed at recommending the most effective variant for real deployment scenarios.

4.4 Test Scenarios and Parameters

Testing is performed using both synthetic datasets and real-world LiDAR scans. Synthetic scans are used to validate correctness under known transformations, while real LiDAR frames simulate practical deployment conditions. Due to ongoing hardware repairs, full system testing with the Bender robot is pending.

Test cases include both translation and rotation, and performance is measured using alignment error and runtime per frame. While comprehensive parameter sweeps and statistical robustness checks are desirable, they are currently limited due to time constraints. However, the system is built to easily support such evaluations in future work.

5 Results

5.1 Accuracy and Drift Over Time

5.2 Runtime Performance

5.3 Convergence Behavior

5.4 Hardware-Specific Observations

6 Conclusion

- Summary of findings
- Recommended ICP variant for lunar robotics
- Future work: robust data association, SLAM loop closure

References

- F. A. Donoso, K. J. Austin, and P. R. McAree. How do icp variants perform when used for scan matching terrain point clouds? *Robotics and Autonomous Systems*, 87:147–161, 2017. doi: 10.1016/j.robot.2016.10.011.
- J. Guevara, J. Gené-Mola, E. Gregorio, and F. A. Auat Cheein. A systematic analysis of scan matching techniques for localization in dense orchards. *Smart Agricultural Technology*, 9:100607, 2024. doi: 10.1016/j.atech.2024.100607.
- Aleksandr Segal, Dirk Haehnel, and Sebastian Thrun. Generalized-icp. In *Robotics: Science and Systems (RSS)*, pages 161–168. MIT Press, 2009. URL <https://doi.org/10.7551/mitpress/8727.003.0022>.

A Programs Used

- Python: NumPy, SciPy, Open3D
- C: Eigen, PlatformIO libraries for Teensy and Raspberry Pi
- Visualization: Matplotlib & Plotly for Python implementation, custom C++ visualization using serial output to python interface